

When you use **asymmetric client authentication (private_key_jwt)** with Epic (or any SMART on FHIR server), the **grant type** depends on the flow you're in:

1. Interactive apps (Authorization Code + PKCE)

- **Grant type:** `authorization_code`
- You authenticate the client at the **token endpoint** using `client_assertion` (the signed JWT).
- Example token request:

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
code=AUTH_CODE_FROM_EPIC
redirect_uri=https://your-app/callback
client_id=YOUR_CLIENT_ID
client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt
client_assertion=SIGNED_JWT
code_verifier=YOUR_PKCE_VERIFIER
```

2. Backend Services (system-to-system apps)

- **Grant type:** `client_credentials`
- You also send `client_assertion` (the signed JWT).
- Example token request:

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
scope=system/Encounter.read
client_id=YOUR_CLIENT_ID
```

```
client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt
client_assertion=SIGNED_JWT
```

3. How to generate `private_key_jwt` in Rails

You'll need:

- A **private key** (usually RSA 2048/3072, in PEM format).
- The **token endpoint URL** (audience claim).
- Your **client_id**.

Example: Using `jwt` gem

Add to Gemfile:

```
gem "jwt"
```

Then in Ruby:

```
require "jwt"
require "securerandom"

private_key = OpenSSL::PKey::RSA.new(File.read("config/private.pem"))

client_id = "YOUR_CLIENT_ID"
token_url = "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token"

now = Time.now.to_i

jwt_payload = {
  iss: client_id,          # issuer = your app
  sub: client_id,          # subject = your app
  aud: token_url,          # audience = token endpoint
  exp: now + 300,          # expires in 5 minutes
  jti: SecureRandom.uuid  # unique ID for this JWT
}

signed_jwt = JWT.encode(jwt_payload, private_key, "RS256")

puts signed_jwt
```

This `signed_jwt` is what you put in the `client_assertion` field when calling the token endpoint.

4. Putting it together

- **Interactive (Auth Code + PKCE)** → `grant_type=authorization_code + client_assertion`
- **Backend (system)** → `grant_type=client_credentials + client_assertion`

Both rely on **private_key_jwt** authentication.

Do you want me to write you a **Rails service object** that does the whole token exchange (build JWT, POST to Epic's token endpoint, return `access_token`)? That way you could just call something like `EpicAuth.exchange_code(auth_code)` or `EpicAuth.client_credentials_token`.